about what a new release contains. For example, we expect a new major version to contain new functionality and API changes incompatible with the previous major version. On the other hand, we would expect a patch to have a limited scope and guarantee backward compatibility.

NOTE If you're working on software projects for which semantic versioning makes sense, I recommend checking out JReleaser, a release automation tool. "Its goal is to simplify creating releases and publishing artifacts to multiple package managers while providing customizable options" (https://jreleaser.org).

Semantic versioning will require some form of manual step to assign a version number based on the content of the release artifact: Does it contain breaking changes? Does it only contain bug fixes? When we have a number, it's still not clear what's included in the new release artifact, so we need to use Git tags and define a mapping between Git commit identifiers and version numbers.

Things get even more challenging for snapshot artifacts. Let's consider a Spring Boot project as an example. By default, we start with version `0.0.1-SNAPSHOT`. Until we're ready to cut the `0.0.1` release, every time we push new changes to the main branch, the commit stage will be triggered, and a new release candidate will be published with the number `0.0.1-SNAPSHOT`. All release candidates will have the same number until version `0.0.1` is released. This approach doesn't ensure traceability of changes. Which commits are included in release candidate `0.0.1-SNAPSHOT`? We can't tell. Furthermore, it's affected by the same unreliability as using `latest`. Any time we retrieve the artifact, it might be different from the last time.

When it comes to continuous delivery, using an approach like semantic versioning is not ideal for uniquely identifying release candidates. When we follow the principles of continuous integration, we'll have many release